

Trabalhando *com condições*

Até aqui, todo programa fazia sempre a mesma coisa. Com `if`, ele **decide**: executa um trecho só quando uma condição é verdadeira. Nesta aula: `if`, `else`, `elif`, condições combinadas, valores "verdadeiros" e "falsos", o operador ternário, o `match` e a importância do recuo.

`if · else · elif`

`Recuo = bloco`

`and · or · in`

`Truthy e falsy`

`x if c else y`

`match/case`

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: `if` Statements, `match` Statements; Truth Value Testing; Conditional expressions), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Executar *só se...*

if_simples.py

```
idade = int(input("Idade: "))
if idade >= 18:
    print("Pode tirar a carteira de motorista.")
    print("(Esta linha também está dentro do if.)")
print("Fim do programa.")  # fora do if: sempre roda
```

SAÍDA (DIGITOU 20)

```
Idade: 20
Pode tirar a carteira de motorista.
(Esta linha também está dentro do if.)
Fim do programa.
```

- ✓ **if condição:** com **dois-pontos** no fim
- ✓ O bloco do if são as linhas com **recuo** (4 espaços)
- ✓ A condição é qualquer coisa que dê **True / False** (aula 06)
- ✓ A linha sem recuo volta ao fluxo normal: roda sempre

Com 15, o programa mostraria só "Fim do programa.": o bloco inteiro é pulado.

Senão, *senão se*

`if_else.py` · DIGITOU 15

```
idade = int(input("Idade: "))
if idade >= 18:
    print("Maior de idade")
else:
    print("Menor de idade")
```

```
Idade: 15
Menor de idade
```

`else` pega todos os outros casos. Nunca tem condição.

`elif.py` · DIGITOU 7.5

```
nota = float(input("Nota: "))
if nota >= 9:
    conceito = "A"
elif nota >= 7:
    conceito = "B"
elif nota >= 5:
    conceito = "C"
else:
    conceito = "D"
print("Conceito:", conceito)
```

```
Nota: 7.5
Conceito: B
```

O Python testa **de cima para baixo** e para no primeiro verdadeiro. Por isso a ordem importa: com `nota >= 5` primeiro, uma nota 9,5 viraria "C".

Condições *combinadas*

combinadas.py

```
idade = 20
tem_ingresso = True
if idade >= 18 and tem_ingresso:
    print("Pode entrar")
dia = "sábado"
if dia == "sábado" or dia == "domingo":
    print("Fim de semana")
if dia in ("sábado", "domingo"):    # o mesmo, mais curto
    print("Fim de semana (com in)")
```

SAÍDA

```
Pode entrar
Fim de semana
Fim de semana (com in)
```

- ✓ `and` : as duas precisam ser verdadeiras; `or` : basta uma (aula 06)
- ✓ `in` com uma tupla substitui vários `or`
- ✓ `18 <= idade < 60` : comparação encadeada, como na matemática

Armadilha: `if dia == "sábado" or "domingo":` é **sempre** verdadeiro, porque o texto "domingo" sozinho conta como True. Compare cada um, ou use `in` .

O que conta como *verdadeiro*

truthy.py · SÓ TECLOU ENTER

```
nome = input("Nome (pode deixar vazio): ").strip()
if nome:
    print(f"Olá, {nome}!")
else:
    print("Olá, visitante!")
```

```
Nome (pode deixar vazio):
Olá, visitante!
```

VALOR	NA CONDIÇÃO VALE
False, None	Falso
0, 0.0	Falso
"", [], (), {}, set()	Falso (vazio)
Qualquer outra coisa	Verdadeiro

Dica do guia (cap. 5): escreva `if lista:` em vez de `if len(lista) > 0:` .

Ternário *e match*

ternario.py

```
idade = 16
status = "adulto" if idade >= 18 else "menor"
print(status)
```

menor

`valor_se_sim if condição else valor_se_não` : um if numa linha, que devolve um valor. Bom para escolhas simples.

match.py · DIGITOU "PARAR"

```
comando = input("Comando: ").strip().lower()
match comando:
    case "iniciar":
        print("Iniciando...")
    case "pausar" | "parar":
        print("Pausando...")
    case _:
        print("Comando desconhecido")
```

Comando: parar
Pausando...

`match` (Python 3.10+) compara um valor com vários casos. | junta opções; `case _:` é o "qualquer outro".

Recuo *esquecido*

erro_recuo.py

```
idade = 20
if idade >= 18:
print("Maior de idade")
```

```
File "erro_recuo.py", line 3
    print("Maior de idade")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

- ✓ Depois de `if ...:`, o Python **espera** um bloco com recuo
- ✓ A mensagem diz exatamente isso: "expected an indented block after 'if' statement on line 2"
- ✓ Padrão: **4 espaços**. O Cursor coloca sozinho ao teclar Enter depois dos dois-pontos
- ✓ Não misture Tab e espaços no mesmo arquivo

Esquecer os **dois-pontos** dá outro erro: `SyntaxError: expected ':'`.

Classificando o *IMC*

Enunciado: peça um IMC (aceitando vírgula) e mostre a faixa: abaixo de 18,5 "abaixo do peso"; até 25 "peso normal"; até 30 "sobrepeso"; acima "obesidade".

Uma resposta possível

```
# Exercício: classificação do IMC
imc = float(input("IMC: ").replace(",", "."))
if imc < 18.5:
    faixa = "abaixo do peso"
elif imc < 25:
    faixa = "peso normal"
elif imc < 30:
    faixa = "sobrepeso"
else:
    faixa = "obesidade"
print(f"IMC {imc:.1f}: {faixa}")
```

SAÍDA (DIGITOU 27,3)

```
IMC: 27,3
IMC 27.3: sobrepeso
```

- ✓ Cada **elif** só precisa do limite de cima: os de baixo já foram descartados
- ✓ Teste os limites: 18.5, 25 e 30 caem na faixa certa?
- ✓ Desafio: junte com a calculadora de IMC da aula 04

Teste o que *ficou*

1. O que marca as linhas que pertencem ao if?

A) Chaves { }

B) O recuo

C) Ponto e vírgula

D) A palavra end

B. 4 espaços por nível.

2. Com nota 9.5 e o elif nota >= 5 vindo primeiro:

A) A

B) C

C) Erro

D) Nada

B. Para no primeiro verdadeiro.

3. Qual destes é falso numa condição?

A) "0"

B) [0]

C) ""

D) -1

C. Texto vazio.

4. "par" if 4 % 2 == 0 else "ímpar" vale:

A) "par"

B) "ímpar"

C) True

D) Erro

A. Operador ternário.

O programa *agora decide*

`if`, `elif` e `else` escolhem o que executar; o bloco é marcado pelo **recuo**. A ordem dos `elif` importa, valores vazios contam como falso, o ternário resolve escolhas simples e o `match` compara um valor com vários casos.

`if · elif · else`

`4 espaços`

`Ordem importa`

`if lista:`

`match/case`

Próxima aula → 14 · Utilizando For

Fontes: docs.python.org/3/tutorial/controlflow.html (`if`, `match`); docs.python.org/3/library/stdtypes.html#truth-value-testing; docs.python.org/3/reference/expressions.html#conditional-expressions, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.